

MixBytes()

Lido Easy Track Factories Security Audit Report

JULY 24, 2026

Table of Contents

1. Introduction	3
1.1 Disclaimer	3
1.2 Executive Summary	3
1.3 Project Overview	5
1.4 Security Assessment Methodology	8
1.5 Risk Classification	11
1.6 Summary of Findings	12
2. Findings Report	14
2.1 Critical	14
2.2 High	14
2.3 Medium	14
M-1 SettleGeneralDelayedPenalty - stale motions can settle a later, unrelated penalty lock for the same operator	14
2.4 Low	16
L-1 AllowConsolidationPair - no per-creator/per-pair limit lets an authorized operator exhaust the global motion slots (Easy Track-wide DoS)	16
L-2 ReportWithdrawalsForSlashedValidators - slashingPenalty is forwarded with only a positivity check and burned verbatim against the operator's bond	18
L-3 SettleGeneralDelayedPenalty - _validateInputData accepts unsorted/duplicate nodeOperatorIds with no ordering or uniqueness guard	20
L-4 ReportWithdrawalsForSlashedValidators - _validateInputData does not verify on-chain slashed status or bound keyIndex, and ERROR_VALIDATOR_NOT_SLASHED is misleading	21
L-5 CreateOrUpdateOperatorGroup - inaccurate module NatSpec and a per-iteration external-module lookup that can be hoisted	22
L-6 CreateOrUpdateOperatorGroup - missing cross-group membership pre-check produces motions that always revert at enactment	23
L-7 UpdateStakingModuleShareLimits - new share values are not bounded by TOTAL_BASIS_POINTS	24

L-8 CreateOrUpdateOperatorGroup - no optimistic-concurrency guard allows a lost update on concurrent group motions	25
L-9 AllowConsolidationPair - allowlist entries are permanent and not re-validated at consolidation execution time	26
3. About MixBytes	27

1. Introduction

1.1 Disclaimer

The audit makes no statements or warranties regarding the utility, safety, or security of the code, the suitability of the business model, investment advice, endorsement of the platform or its products, the regulatory regime for the business model, or any other claims about the fitness of the contracts for a particular purpose or their bug-free status.

1.2 Executive Summary

Easy Track is Lido's optimistic-governance framework: rather than running a full Aragon vote, a proposer creates a time-boxed motion that executes automatically unless LDO holders raise objections above a threshold during a delay window. The six in-scope contracts are EVMScript factories supporting the upcoming Staking Router v3 (SRv3), Curated Module v2 (CMv2), and Community Staking Module v3 (CSMv3) releases – each validates a narrow set of operator or governance inputs and emits a single, pre-authorized call, such as updating staking-module share limits, creating or updating MetaRegistry operator groups, allowing validator consolidation pairs, reporting slashed/withdrawn validators, settling delayed penalties, or setting a gate's Merkle tree.

The interim security review was conducted over 10 working days via manual code review and a proprietary AI-assisted analysis tool.

Because each factory's purpose is to gate calls into out-of-scope target contracts, our review deliberately spanned both the in-scope validation logic and the way each produced EVMScript is consumed by those external integrations – the Staking Router, ConsolidationMigrator, MetaRegistry, CSM modules, and Merkle gates. Alongside this, we worked through our standard internal checklist: access-control and trusted-caller gating, unchecked or attacker-influenced inputs, integer truncation and unsafe casts, rounding, and the consistency between motion creation and enactment. The factory-specific vectors we examined are listed below.

- **EVMScript target and permission scoping.** We verified that each factory emits calls only to its intended target and selector and that the resulting script is constrained by the registry permission allowlist.
- **Array de-duplication and ordering.** We checked that batch inputs are strictly ordered and de-duplicated to prevent double-processing, comparing each factory against its siblings; [SettleGeneralDelayedPenalty](#) omits the strict-ascending check the others enforce (benign against the idempotent target, but a consistency gap).
- **Privileged parameter bounds.** We reviewed [UpdateStakingModuleShareLimits](#) for per-motion delta caps versus an absolute basis-point ceiling and the role of the embedded [validateParams](#) re-validation call relative to the Staking Router's own on-chain clamps.
- **Trusted-input vs on-chain state.** We assessed whether committee-attested figures are cross-checked against on-chain state, notably [ReportWithdrawalsForSlashedValidators](#), which validates a self-reported [isSlashed](#) flag and an unbounded [slashingPenalty](#) magnitude (see finding) while the production CSM module independently requires an on-chain slashing proof and prevents double-reporting. The factory does not itself verify the slashing was confirmed on-chain via

the verifier; it trusts the caller's `isSlashed` flag and defers that check to the module at enactment. We also examined the conditions the factory leaves to enactment-time re-validation rather than checking at creation – a typo'd `keyIndex` or a key not yet on-chain-flagged as slashed makes a motion revert at enactment instead of at creation; the module's enactment re-check keeps this fail-safe, so we treated it as a fail-fast/UX gap rather than a vulnerability.

- **Group and share accounting integrity.** We validated operator-group composition rules – share sums, per-element shares, member existence, and source<->target group linkage – in `CreateOrUpdateOperatorGroup` and `AllowConsolidationPair` against the MetaRegistry's own validation. We confirmed that cross-group membership conflicts (a sub or external operator already belonging to another group) are not checked at motion creation but are caught by `MetaRegistry` at enactment, so a misconfigured motion reverts rather than corrupting group state – again a fail-fast gap left fail-safe by enactment re-validation.
- **Motion creation vs enactment consistency.** We confirmed that every factory produces a deterministic EVMScript that is a pure function of call data, so the hash-checked re-creation at enactment cannot silently diverge.

The target contracts these factories drive – the Staking Router, ConsolidationMigrator, MetaRegistry, the CSM and curated staking modules together with their Accounting, and the Merkle gates – along with the core Easy Track machinery (`EasyTrack`, `EVMScriptExecutor`, the factories registry, and the Aragon `CallsScript` executor) were not themselves part of this engagement; we examined only how the in-scope factories call into them, cross-referencing the relevant production sources and interfaces rather than auditing those contracts line by line.

Below we set out our overall assessment, key assumptions, and main recommendations.

- **Clear, consistent factory pattern.** All six contracts follow the established Easy Track `IEVMScriptFactory` shape – decode, validate, emit a single deterministic script – which keeps them small and easy to reason about. Inputs are checked with explicit revert reasons, and the produced scripts are pure functions of the call data, so motion creation and enactment remain consistent.
- **Validation is largely defense-in-depth.** The privileged targets independently re-validate the same constraints on-chain (Staking Router share bounds, MetaRegistry share-sum and group rules, CSM slashing/withdrawal guards), so the factory checks reinforce target guarantees rather than being the sole line of defense. This is positive for robustness, but it also means a missing factory check generally degrades to a fail-fast or UX gap rather than a vulnerability.
- **Minor validation-consistency gaps.** A few factories diverge from their siblings – e.g. `SettleGeneralDelayedPenalty` omits the strict-ascending de-duplication used elsewhere, and `CreateOrUpdateOperatorGroup` permits per-element zero shares while validating only the aggregate sum. These are low-impact but worth aligning for consistency.
- **Authorization/consent surface in the operator-authorized factory.** `AllowConsolidationPair` lets an authorized caller choose – and later overwrite – the consolidation submitter; binding the submitter to a consenting party and checking existing allowlist state would tighten the consent model.
- **Shared-availability consideration.** The global 24-motion cap is shared across all factories; operator-callable factories can occupy those slots. This is an accepted Easy Track property, but it is worth noting for the newly permissionless consolidation factory.
- **Stale-motion accumulation against the shared cap.** Easy Track motions do not expire: a motion that reverts at enactment (e.g. one whose current-state snapshot no longer matches after the re-audit's nonce/group-hash guards) keeps its slot until manually cancelled. Such "zombie"

motions accumulate and push the shared `motionsCountLimit` toward exhaustion – a primarily operational risk that becomes material during bulk operations such as a mass group migration.

1.3 Project Overview

Summary

Title	Description
Client	Lido
Category	Liquid Staking
Project	Easy Track Factories (SRv3/CMv2)
Type	Solidity
Platform	EVM
Timeline	27.05.2026 – 16.06.2026

Scope of Audit

File	Link
<code>contracts/EVMScriptFactories/AllowConsolidationPair.sol</code>	contracts/EVMScriptFactories/AllowConsolidationPair.sol
<code>contracts/EVMScriptFactories/CreateOrUpdateOperatorGroup.sol</code>	contracts/EVMScriptFactories/CreateOrUpdateOperatorGroup.sol
<code>contracts/EVMScriptFactories/ReportWithdrawalsForSlashedValidators.sol</code>	contracts/EVMScriptFactories/ReportWithdrawalsForSlashedValidators.sol
<code>contracts/EVMScriptFactories/SetMerkleGateTree.sol</code>	contracts/EVMScriptFactories/SetMerkleGateTree.sol
<code>contracts/EVMScriptFactories/SettleGeneralDelayedPenalty.sol</code>	contracts/EVMScriptFactories/SettleGeneralDelayedPenalty.sol
<code>contracts/EVMScriptFactories/UpdateStakingModuleShareLimits.sol</code>	contracts/EVMScriptFactories/UpdateStakingModuleShareLimits.sol

Versions Log

Date	Commit Hash	Note
27.05.2026	9fedee6aede0898d2c924e4b99ce688a352dd11c	Initial Commit
16.06.2026	b5d74557448a7f65ea74abdc09ff954bd113c8c3	Commit for Re-audit

Mainnet Deployments

File	Address
SetMerkleGateTree.sol (CSM)	0xf3ec30B86c3dC1b8a1C754D885F9bE3160e15B4c
SetMerkleGateTree.sol (CM)	0xa121667D1780a1D54EAE67AE17ee13d0f872D60
ReportWithdrawalsForSlashedValidators.sol (CSM)	0xE330516a03bDdEBA4209b5591112f1aa3dd90F0A
ReportWithdrawalsForSlashedValidators.sol (CM)	0x71862Abd99819597670007bb992A7a7562fE50f2
SettleGeneralDelayedPenalty.sol (CSM)	0xB71755bE764abB4Ce26cb4dADf056Be57fB8880F
SettleGeneralDelayedPenalty.sol (CM)	0xffffEFC16231eDC6Dc9C93e364ff4D4E3f787f416
CreateOrUpdateOperatorGroup.sol (CM)	0x2fC78638b77381e9D040163Bd6EB1cac967bDBdF
AllowConsolidationPair.sol	0x29e23B1EF0c9ffffAc8330F9abaCebDDD827E4b5C
UpdateStakingModuleShareLimits.sol (CSM)	0xde3e46E3129fA4e4e3f66c9024B0A3Ad509b27a1

All nine deployments are in scope: each is an instance of one of the six audited factories (the CSM/CM suffix denotes the module the instance is bound to, not a different contract). Every deployment compiles byte-for-byte from the audit-scope source at commit

[b5d74557448a7f65ea74abdc09ff954bd113c8c3](#) (solc 0.8.6, EVM version [berlin](#), optimizer disabled), with the only deviations being the constructor-immutable slots, which hold the expected values. Dependencies are pinned. The full transitive import closure of all six factories consists of 17 in-repository files only (the factory, [TrustedCaller](#), [EVMScriptCreator](#), and local interfaces); none pulls in an external library, so no third-party code is compiled into the deployed bytecode. The repository's sole external dependency, OpenZeppelin Contracts, is pinned to an exact version (4.3.2, vendored under [dependencies/](#) and remapped in [brownie-config.yaml](#), with no floating version range), but it is used only by out-of-scope Easy Track core contracts and is not reachable from any of the deployed factories.

Each deployment's constructor arguments were cross-checked against the published deployment manifests ([deployed-sm-mainnet.json](#) / [deployed-sr-mainnet.json](#)) and the on-chain immutables read back from the contracts; all match exactly. For eight of the nine factories the reference is the manifest in [lido/finance/easy-track#125](#); for the corrected CSM [UpdateStakingModuleShareLimits](#)

instance the reference is the updated manifest in [lidofinance/easy-track#128](#) (the original #125 entry for that factory held the misconfigured caps described below). The immutable configuration is additionally internally consistent and resolves to the expected Lido contracts on the canonical StakingRouter [0xFdDf38947aFB03C621C71b06C9C70bce73f12999](#): for each [SettleGeneralDelayedPenalty](#) instance the `accounting` immutable equals `module.ACCOUNTING()`; the [CreateOrUpdateOperatorGroup](#) module, `metaRegistry` and `stakingRouter` form a consistent bidirectional linkage (`curatedModule.META_REGISTRY() == metaRegistry`, `metaRegistry.STAKING_ROUTER() == stakingRouter`) with `allowedExternalModuleId = 1`; the [UpdateStakingModuleShareLimits](#) instance targets `stakingModuleId = 3` with per-motion caps `maxStakeShareLimitIncrease/Decrease = 50 bp (0.5%)` and `maxPriorityExitShareThresholdIncrease/Decrease = 60 bp (0.6%)`; and the [AllowConsolidationPair](#) `consolidationMigrator`, `stakingRouter`, `sourceModuleId = 1` and `targetModuleId = 4` match the migrator's own getters.

Corrected [UpdateStakingModuleShareLimits \(CSM\) redeployment](#). The [UpdateStakingModuleShareLimits \(CSM\)](#) factory listed above, [0xde3e46E3129fA4e4e3f66c9024B0A3Ad509b27a1](#), is a corrected redeployment. The originally deployed instance, [0x0C6703F1d8D9DdFB6c6e5F57b4f7432a6500D6D8](#), was configured with per-motion caps that were 10x larger than intended: `maxStakeShareLimitIncrease/Decrease = 500 bp` and `maxPriorityExitShareThresholdIncrease/Decrease = 600 bp` – i.e. 5.0% and 6.0% rather than the intended 0.5% and 0.6%. The discrepancy traces to a basis-point misreading in the source parameter proposal (500 bp being 5% rather than the stated 0.5%). Because the error went unnoticed, the misconfigured factory was already registered in the Easy Track factory registry by the earlier on-chain Vote #203.

The remediation is a governance-level correction rather than a code change: the factory contract's logic is identical (the caps are constructor immutables, not encoded logic), so the fix is a redeployment with the correct arguments plus a registry swap. The corrected factory [0xde3e46E3129fA4e4e3f66c9024B0A3Ad509b27a1](#) is proposed to be added to the Easy Track registry with unchanged permissions while the misconfigured [0x0C6703F1d8D9DdFB6c6e5F57b4f7432a6500D6D8](#) is removed, both in the next on-chain vote. We verified on-chain that the two instances share identical runtime bytecode aside from the immutable parameter slots, that the corrected instance reads back `maxStakeShareLimitIncrease/Decrease = 50 bp` and `maxPriorityExitShareThresholdIncrease/Decrease = 60 bp`, and that all other configuration is unchanged from the original – same `trustedCaller` (the CSM committee [Safe 0xC52fC3081123073078698F1EAc2f1Dc7Bd71880f](#)), same `stakingRouter` [0xFdDf38947aFB03C621C71b06C9C70bce73f12999](#), and same `stakingModuleId = 3`. The corrected deployment manifest is published in [lidofinance/easy-track#128](#).

Access control is enforced through the immutable `trustedCaller` on each factory; [AllowConsolidationPair](#) is permissionless by design. The two committees configured as `trustedCaller` are both Gnosis Safe multisigs: the CSM committee [0xC52fC3081123073078698F1EAc2f1Dc7Bd71880f](#) is a Safe v1.3.0 (canonical singleton [0xd9Db270c1B5E3Bd161E8c8503c55cEABeE709552](#)) with a 4-of-6 threshold, and the CM committee [0x2570e0b22AD904501dFB0d49575991ACB801dD91](#) is a Safe v1.4.1 with a 5-of-9 threshold.

1.4 Security Assessment Methodology

Stage 1: Interim Audit

Project Architecture Review

OBJECTIVE: UNDERSTAND THE OVERALL STRUCTURE OF THE PROTOCOL AND IDENTIFY POTENTIAL SECURITY RISKS, INCLUDING PRIMITIVES AND ABSTRACTIONS IMPLEMENTED BY THE SYSTEM, HOW THEY INTERACT ARCHITECTURALLY, AND WHERE DESIGN OR INTEGRATION WEAKNESSES COULD BE EXPLOITED.

Activities include:

- understanding overall system design and contract interactions
- mapping responsibilities of each component and protocol workflows
- conducting a thorough line-by-line review of contracts and modules
- tracing execution paths and mapping state transitions
- identifying trust boundaries and external integrations
- forming an independent architectural understanding directly from code before validating documentation
- running the project test suite to observe implementation behavior and encoded invariants
- reviewing documentation, specifications, READMEs, and deployment guides and reconciling them with code

Adversarial Code Review

OBJECTIVE: IDENTIFY POTENTIAL VULNERABILITIES SPECIFIC TO THE PROTOCOL ARCHITECTURE, BUSINESS LOGIC, AND ECONOMIC MECHANISMS.

Approach includes:

- analyzing the codebase from an attacker's perspective, focusing on what can fail rather than only confirming intended behavior
- searching for overlooked edge cases, invalid assumptions, unexpected call sequences, and unsafe state transitions
- attempting to violate protocol invariants and identifying scenarios where system guarantees break
- analyzing economic attack surfaces including liquidation logic, oracle dependencies, vault accounting, and cross-contract interactions
- writing targeted tests, modelling adversarial scenarios, applying mutation testing and fuzzing, and developing proof-of-concept exploits
- conducting independent exploit path discovery by each auditor to reduce anchoring bias
- reviewing high-risk code collaboratively in pair-auditing sessions led by the Team Lead

Systematic Vulnerability Analysis

OBJECTIVE: APPLY STRUCTURED ANALYSIS AND KNOWN ATTACK PATTERNS TO ENSURE COMPREHENSIVE COVERAGE ACROSS THE CODEBASE.

Activities include:

- reviewing code against an internally maintained vulnerability checklist derived from past exploits and research
- analyzing common DeFi attack classes such as reentrancy, accounting manipulation, oracle manipulation, privilege escalation, and state desynchronization
- using proprietary AI tooling to surface candidate issues across broader attack vectors
- manually validating and triaging AI-generated candidates

Consolidation of Auditors' Reports

OBJECTIVE: MERGE INTERIM INPUTS FROM ALL AUDITORS INTO A COHERENT REPORT WITH CONSISTENT SEVERITY LEVELS AND REPRODUCIBLE FINDINGS.

Activities include:

- cross-checking findings across auditors
- consolidating and deduplicating issues
- resolving discrepancies in severity assessments
- using proprietary AI tooling to assist with report drafting and consistency checks
- manually reviewing and finalizing the report narrative
- preparing the interim audit report for client review

Stage 2: Re-audit & Mainnet Deployment Verification

Re-audit

OBJECTIVE: VERIFY THAT CLIENT REMEDIATIONS ARE CORRECTLY IMPLEMENTED AND THAT THE UPDATED CODE DOES NOT INTRODUCE NEW VULNERABILITIES.

Activities include:

- confirming each remediation matches the original recommendation
- documenting rationale for any unresolved findings
- verifying modified logic and integration points remain secure
- executing the project's tests after remediation, including targeted coverage of fixed areas
- using proprietary AI tooling on the updated codebase to surface potential new issues

Mainnet Deployment Verification

OBJECTIVE: PERFORM FINAL VERIFICATION OF DEPLOYED CONTRACTS AND CONFIGURATION BEFORE ISSUING THE PUBLIC AUDIT REPORT.

Activities include:

- verifying deployed contract bytecode against audited source code across networks
- confirming deployed bytecode matches the build produced from the audited commit using identical compiler settings
- reviewing constructor and initializer arguments used in deployment
- verifying proxy deployment order, initialization flow, and admin configuration
- ensuring implementations are not left uninitialized or exposed to initialization front-running
- publishing the final report after alignment between client and audit team

1.5 Risk Classification

Severity Level Matrix

Severity	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

Impact

- **High** – Theft exceeding 0.5% of the protocol's TVL, partial or complete blocking of funds on the contract without the possibility of withdrawal (>0.5%), or loss of user funds exceeding 1% for users interacting with the protocol.
- **Medium** – Contract lock that can only be resolved through a contract upgrade, one-time theft of rewards or an amount up to 0.5% of the protocol's TVL, or funds lock where withdrawal is still possible by an administrator.
- **Low** – One-time contract lock that can be resolved by an administrator without requiring a contract upgrade.

Likelihood

- **High** – An event with an estimated 50-60% probability of occurring within a year, which can be triggered by any actor (e.g., due to a market condition that the actor cannot influence).
- **Medium** – An unlikely event (10-20% probability of occurring) that can be triggered by a trusted actor.
- **Low** – A highly unlikely event that can only be triggered by the contract owner.

Action Required

- **Critical** – Must be fixed as soon as possible.
- **High** – Strongly advised to be fixed in order to minimize potential risks.
- **Medium** – Recommended to be fixed to improve security and stability.
- **Low** – Recommended to be fixed to improve overall robustness and efficiency.

Finding Status

- **Fixed** – The recommended fixes have been implemented in the project code and the issue no longer impacts the security of the protocol.
- **Partially Fixed** – The recommended fixes have been partially implemented, reducing the impact of the finding, but the issue has not been fully resolved.
- **Acknowledged** – The recommended fixes have not been implemented, and the finding remains unresolved or has been accepted by the project team without code changes.

1.6 Summary of Findings

Findings Count

Severity	Count
Critical	0
High	0
Medium	1
Low	9

Findings Statuses

ID	Finding	Severity	Status
M-1	SettleGeneralDelayedPenalty - stale motions can settle a later, unrelated penalty lock for the same operator	Medium	Fixed
L-1	AllowConsolidationPair - no per-creator/per-pair limit lets an authorized operator exhaust the global motion slots (Easy Track-wide DoS)	Low	Acknowledged
L-2	ReportWithdrawalsForSlashedValidators - <code>slashingPenalty</code> is forwarded with only a positivity check and burned verbatim against the operator's bond	Low	Acknowledged
L-3	SettleGeneralDelayedPenalty - <code>_validateInputData</code> accepts unsorted/duplicate <code>nodeOperatorIds</code> with no ordering or uniqueness guard	Low	Fixed
L-4	ReportWithdrawalsForSlashedValidators - <code>_validateInputData</code> does not verify on-chain slashed status or bound <code>keyIndex</code> , and <code>ERROR_VALIDATOR_NOT_SLASHED</code> is misleading	Low	Fixed
L-5	CreateOrUpdateOperatorGroup - inaccurate <code>module NatSpec</code> and a per-iteration external-module lookup that can be hoisted	Low	Fixed

L-6	CreateOrUpdateOperatorGroup - missing cross-group membership pre-check produces motions that always revert at enactment	Low	Acknowledged
L-7	UpdateStakingModuleShareLimits - new share values are not bounded by TOTAL_BASIS_POINTS	Low	Fixed
L-8	CreateOrUpdateOperatorGroup - no optimistic-concurrency guard allows a lost update on concurrent group motions	Low	Fixed
L-9	AllowConsolidationPair - allowlist entries are permanent and not re-validated at consolidation execution time	Low	Acknowledged

2. Findings Report

2.1 Critical

Not Found

2.2 High

Not Found

2.3 Medium

M-1	SettleGeneralDelayedPenalty - stale motions can settle a later, unrelated penalty lock for the same operator		
Severity	Medium	Status	Fixed in b5d74557

Description

`SettleGeneralDelayedPenalty._validateInputData` validates only that each `maxAmount` is non-zero and greater than or equal to the current aggregated locked bond for the corresponding Node Operator. EasyTrack then calls `module.settleGeneralDelayedPenalty(nodeOperatorIds, maxAmounts)` and does not bind the motion to a specific penalty report, lock timestamp, or expected locked amount.

If the original lock is compensated, cancelled, settled, or expires, and a new penalty lock for the same Node Operator appears before the old motion is enacted, the old motion can still pass validation as long as the new `lockedBond` is less than or equal to the stale `maxAmount`. The downstream module/accounting logic settles the current aggregated lock by burning `min(currentLockedBond, maxAmount)`, so the stale motion can burn bond for a penalty that was not the one reviewed when the motion was created.

If the current locked amount becomes greater than `maxAmount`, `enactMotion()` reverts because the factory re-validation fails. However, Easy Track motions do not expire after becoming enactable. As a result, such a motion may remain indefinitely in the active motions list while it is temporarily impossible to execute. Later, if the locked amount is replaced by another lock whose amount is less than or equal to `maxAmount`, anyone can permissionlessly call `enactMotion()` and execute the stale motion against the then-current lock.

Code reference: `SettleGeneralDelayedPenalty._validateInputData`.

This issue is classified as Medium severity because it can bypass the intended fresh Easy Track objection window for an irreversible bond burn, while the impact is bounded by `maxAmount`, limited to the same Node Operator, and requires trusted/permissioned actors to create the motion or report the penalty, per the Impact/Likelihood matrix.

Recommendation

We recommend binding settlement motions to the reviewed lock rather than only to a maximum amount. Preferably, introduce a lock nonce or penalty ID in the module and require it during settlement. As a weaker mitigation, include the expected lock expiry time (`until`) and require the current values to match during both motion creation and enactment.

Client's Commentary:

Client: Fixed as of [b5d74557448a7f65ea74abdc09ff954bd113c8c3](#)

MixBytes: The fix spans two repositories. In addition to the Easy Track changes in [b5d74557448a7f65ea74abdc09ff954bd113c8c3](#), we also reviewed the module-side counterpart in [lidofinance/staking-modules#816](#), which adds the `getBondLockNonce` getter consumed by the factory and increments the bond-lock nonce on each new lock (while leaving it unchanged on lock removal/compensation), so a stale motion bound to a superseded lock reverts at settlement with `InvalidBondLockNonce`. With both halves in place, the finding is resolved.

2.4 Low

L-1	AllowConsolidationPair - no per-creator/per-pair limit lets an authorized operator exhaust the global motion slots (Easy Track-wide DoS)		
Severity	Low	Status	Acknowledged

Description

`AllowConsolidationPair` is the only in-scope factory without a `TrustedCaller` gate; instead it authorizes `_creator` as the source operator's reward address or a holder of `MANAGE_SIGNING_KEYS_ROLE` for that operator. It imposes no per-creator, per-pair, or cooldown limit, so a single authorized curated operator can repeatedly call `EasyTrack.createMotion` with valid inputs (reusing the same in-group source->targets, varying `submitter`, etc.). Easy Track enforces one `global active-motion cap` shared by every factory: `require(motions.length < motionsCountLimit)` with `MAX_MOTIONS_LIMIT = 24`, and performs no per-creator, per-factory, or duplicate (`evmScriptHash`) checks. Once the array is full, all `createMotion` calls revert - including trusted-committee motions for share-limit updates, slashing reports, and penalty settlement. A spam motion holds its slot until it is enacted (only after `motionDuration >= 48h`), objected past the threshold, or cancelled via `CANCEL_ROLE`; the spammer cannot be evicted sooner. Thus one misbehaving or compromised authorized operator can halt Easy Track motion processing system-wide for `>=48h` at a time.

Because `cancelMotion` is creator-only and has no timelock, the same property also enables a front-run-and-cancel variant: an attacker observing a victim's `createMotion` in the public mempool can fill the queue so the victim's call reverts with `MOTIONS_LIMIT_REACHED`, then immediately cancel its own motions, leaving a clean current-state snapshot (the `MotionCreated/MotionCanceled` events remain permanently visible on-chain and point to `motion.creator`). Routing critical governance motions through private orderflow defeats this vector.

This is largely an architecturally-accepted property of Easy Track's shared motion limit rather than a defect introduced by this release; the same property applies to the pre-existing `IncreaseNodeOperatorStakingLimit` / `IncreaseVettedValidatorsLimit` / `CuratedSubmitExitRequestHashes` factories. Mitigations: recoverable without an upgrade via `cancelAllMotions/cancelMotions` (`CANCEL_ROLE`), `pause`, or `removeEVMScriptFactory`; governance can also act directly through the DAO Agent bypassing Easy Track entirely; `>=48h` auto-expiry; gas cost per motion; and the fact that callers are vetted curated operators with publicly attributable reward addresses.

This issue is classified Low severity because recovery requires only an admin/`CANCEL_ROLE` action without a contract upgrade and the trigger is a semi-trusted vetted actor, per the Impact/Likelihood matrix.

Recommendation

We recommend confirming that extending the operator-callable (non-`TrustedCaller`) motion pattern to consolidation is an accepted risk under the existing `MAX_MOTIONS_LIMIT/CANCEL_ROLE` model, and - if tighter isolation is desired - considering a per-creator active-motion cap (or per-source-operator dedup), a duplicate-motion guard, and/or reserving part of `motionsCountLimit` for trusted-caller factories so a single operator cannot monopolize the shared 24-slot motion array. Critical governance motions should additionally be submitted via private orderflow rather than the public mempool.

Client's Commentary:

Acknowledged. Curated node operators are considered trusted entities, and the DAO would use different leverage in the case of misbehaviour.

L-2	ReportWithdrawalsForSlashedValidators - <code>slashingPenalty</code> is forwarded with only a positivity check and burned verbatim against the operator's bond		
Severity	Low	Status	Acknowledged

Description

`ReportWithdrawalsForSlashedValidators._validateInputData` checks each entry only for `nodeOperatorId < nosCount`, `exitBalance > 0`, `isSlashed`, `slashingPenalty > 0`, and strict `(nodeOperatorId, keyIndex)` ordering. There is no upper bound on `slashingPenalty`, and the factory does not cross-check the magnitude against any on-chain value.

The factory assumes the caller-supplied magnitude is correct, and when a non-zero `slashingPenalty` is supplied nothing downstream re-bounds it. The emitted script calls `module.reportSlashedWithdrawnValidators`, whose `_fulfillExitObligations` adds `validatorInfo.slashingPenalty` verbatim to `penaltySum` ("Slashing penalty doesn't scale because all the losses are already accounted") and calls `accounting.penalize(nodeOperatorId, penaltySum)`, which burns up to the operator's full bond and pushes any remainder into `bondDebt`. The supplied magnitude is unbounded for all practical purposes – the only module-side guard is an overflow revert around 2^{245} wei.

The one genuine downstream control gates eligibility, not amount: the `isSlashed` entry reverts with `SlashingPenaltyIsNotApplicable` unless `$.isValidatorSlashed[pointer]` was already set by `reportValidatorSlashing`, which is `VERIFIER_ROLE`-gated and carries an on-chain (EIP-4788 beacon-root) slashing proof. That witness proves the key was genuinely slashed – it gates whether a key is penalizable, not how much. The magnitude itself is forwarded with no bound at all.

Per LIP-33 the real penalty is physically bounded by validator size (pre-slashing balance minus withdrawn plus missed rewards, at most `MAX_EFFECTIVE_BALANCE` = 2048 ETH per key). A trusted-committee fat-finger or a compromised committee key can therefore submit an arbitrarily large penalty for a key that is legitimately flagged `isValidatorSlashed`, with only the objection window as a control.

Code reference: `ReportWithdrawalsForSlashedValidators._validateInputData`.

Exploit scenario:

1. The trusted caller encodes `WithdrawnValidatorInfo{nodeOperatorId: 3, keyIndex: 0, exitBalance: 32e18, slashingPenalty: 100e18, isSlashed: true}` for a key legitimately flagged `isValidatorSlashed` whose real penalty is ~1 ETH (the inflated `slashingPenalty` is independent of `exitBalance`; both are caller-attested).
2. `_validateInputData` passes (positivity and ordering only) and the motion clears the objection window.
3. Enactment calls `reportSlashedWithdrawnValidators -> _fulfillExitObligations` (adding 100e18 to `penaltySum`) `-> accounting.penalize -> _burn`, burning the operator's full bond and pushing the remainder into `bondDebt`.

Impact: an honest node operator can have their entire bond burned beyond the economically-justified slashing loss. Every harm path requires the trusted committee multisig holding `REPORT_SLASHED_WITHDRAWN_VALIDATORS_ROLE` to publish an inflated off-chain-derived magnitude for a key that a separate verifier has already proven slashed; the result is irreversible bond destruction bounded by the operator's full bond per key.

This issue is classified Low severity because the trigger is limited to the trusted committee publishing an inflated magnitude and the on-chain effect, while irreversible, is bond destruction recoverable only through governance rather than a value transfer to the attacker, per the Impact/Likelihood matrix.

Recommendation

We recommend adding a factory-level absolute ceiling on `slashingPenalty` per entry - e.g. `require(current.slashingPenalty <= MAX_EFFECTIVE_BALANCE)` (2048 ETH), or a configurable immutable `maxSlashingPenaltyPerKey` set to the same physical bound.

Client's Commentary:

Acknowledged. Since the factory is behind a trusted actor, it seems reasonable to rely on the committee providing the correct values.

L-3	SettleGeneralDelayedPenalty - <code>_validateInputData</code> accepts unsorted/duplicate <code>nodeOperatorIds</code> with no ordering or uniqueness guard		
Severity	Low	Status	Fixed in b5d74557

Description

`SettleGeneralDelayedPenalty._validateInputData` loops over `nodeOperatorIds` checking only per-element constraints - `nodeOperatorId < count`, `maxAmount > 0`, and `maxAmount >= locked`. It has no `nodeOperatorId > prevId` check. This is the only batch factory in scope without the strict-ascending ordering/uniqueness guard that every sibling enforces (the `report factory` and the `operator-group factory`).

The factory assumes downstream settlement behaves correctly for any element list, and today it does: `Accounting.settleLockedBond` settles `amountSettled = min(lockedAmount, maxAmount)` and consumes the lock, so a duplicate id's second occurrence reads `lockedAmount == 0` and settles nothing - there is no double-burn beyond the live lock. The defect is that the factory emits a non-canonical array it cannot verify is idempotent from its own interface, diverging from every sibling.

Code reference: `SettleGeneralDelayedPenalty._validateInputData`.

Impact: today the harm is a latent over-settlement risk if a future module variant settles per-call rather than min-with-lock. No stETH is lost under current downstream semantics.

This issue is classified Low severity because there is no fund loss or compounding under current downstream semantics (idempotent settle) and the only material effect today is a validation-hygiene asymmetry affecting off-chain rendering and objector review, per the Impact/Likelihood matrix.

Recommendation

We recommend adding `if (i > 0) require(nodeOperatorId > prevId, ERROR_NOT_SORTED)`; mirroring the sibling factories, which enforces ordering and uniqueness and makes the emitted array canonical regardless of downstream settle semantics.

Client's Commentary:

Fixed as of b5d74557448a7f65ea74abdc09ff954bd113c8c3

L-4	ReportWithdrawalsForSlashedValidators - <code>_validateInputData</code> does not verify on-chain slashed status or bound <code>keyIndex</code> , and <code>ERROR_VALIDATOR_NOT_SLASHED</code> is misleading		
Severity	Low	Status	Fixed in b5d74557

Description

This issue has been identified within the `_validateInputData` function of the `ReportWithdrawalsForSlashedValidators` contract. The function `validates` the node operator ID, exit balance, slashing status, slashing penalty, and ordering of the submitted validator records, but leaves two enactment-time preconditions unchecked at creation:

1. **On-chain slashed status is not verified.** The `ERROR_VALIDATOR_NOT_SLASHED` check validates only the `isSlashed` flag contained in the `input data`, not the on-chain slashed status. At enactment the module additionally requires that the validator was previously reported slashed on-chain (reverting with `SlashingPenaltyIsNotApplicable`), which the factory never confirms. The error naming is therefore misleading: it may give a false sense that on-chain slashing is being validated, when only a caller-supplied boolean is.
2. **`keyIndex` is not bounded.** The function does not verify that each `keyIndex` is lower than the corresponding node operator's `totalDepositedKeys`. At enactment the module rejects out-of-range indices (`SigningKeysInvalidOffset`). As a result, an EVMScript may be created with a validator key index that does not belong to a deposited validator.

Because a single failing element reverts the entire batch, a committee can wait the full objection window only to have enactment revert, delaying the intended withdrawal report. The target module exposes public getters suitable for a pre-check – `isValidatorSlashed(nodeOperatorId, keyIndex)` and `isValidatorWithdrawn(nodeOperatorId, keyIndex)` – though the in-scope `IBaseModule` interface would need to be extended to expose them to the factory.

Code reference: `ReportWithdrawalsForSlashedValidators._validateInputData`.

This issue is classified as Low severity because it can lead to invalid or unexecutable Easy Track motions rather than direct state corruption – the downstream module rejects both out-of-range key indices and not-yet-slashed keys – but the missing checks waste the objection window and the misleading error naming may give a false sense that on-chain slashing is validated, per the Impact/Likelihood matrix.

Recommendation

We recommend adding per-element validation in `_validateInputData` that checks `module.isValidatorSlashed(...)` (the actual intent behind `ERROR_VALIDATOR_NOT_SLASHED`) and the `keyIndex < totalDepositedKeys` bound, and optionally `!module.isValidatorWithdrawn(...)` to avoid creating no-op motions. This requires extending the `IBaseModule` interface with the corresponding getters.

Client's Commentary:

Fixed as of b5d74557448a7f65ea74abdc09ff954bd113c8c3

L-5	CreateOrUpdateOperatorGroup - inaccurate <code>module</code> NatSpec and a per-iteration external-module lookup that can be hoisted		
Severity	Low	Status	Fixed in b5d74557

Description

Two minor code-quality issues were identified in the `CreateOrUpdateOperatorGroup` contract:

1. The `NatSpec` for the `immutable` is inaccurate. The constructor takes the curated module from the `_module` argument and derives the `MetaRegistry` from it via `curatedModule.META_REGISTRY()` - the curated module is not taken from the `MetaRegistry`, the relationship is the other way around.
2. In `_validateExternalOperators`, the external module lookup `stakingRouter.getStakingModule(externalModuleId).stakingModuleAddress` is performed inside the loop, although `externalModuleId` is required to equal the `immutable allowedExternalModuleId` on every iteration and is therefore constant across the batch. The lookup can be hoisted out of the loop to avoid repeated external calls.

This issue is classified as Low severity because it does not affect the correctness or security of the protocol, but it improves code clarity and reduces avoidable gas overhead, per the Impact/Likelihood matrix.

Recommendation

We recommend correcting the `module` NatSpec to reflect that the `MetaRegistry` is derived from the curated module, and hoisting the external-module address lookup out of the `_validateExternalOperators` loop.

Client's Commentary:

Fixed as of b5d74557448a7f65ea74abdc09ff954bd113c8c3

L-6	CreateOrUpdateOperatorGroup - missing cross-group membership pre-check produces motions that always revert at enactment		
Severity	Low	Status	Acknowledged

Description

This issue has been identified within the `_validateSubNodeOperators` and `_validateExternalOperators` functions of the `CreateOrUpdateOperatorGroup` contract. The factory `validates` that each sub/external operator exists and that the lists are sorted, but it does not check whether an operator is already a member of another group. The target `MetaRegistry` reverts at enactment (`NodeOperatorAlreadyInGroup` for sub-operators, `AlreadyUsedAsExternalOperator` for external operators) when an operator already belongs to a different group. Updating the group's own composition is fine because the registry clears the group's indices first, so the conflict only arises with membership in another group. Consequently, a motion that references an operator already assigned to another group passes factory validation, is created successfully, remains in the active motions list for at least `motionDuration` (~48h), and is then guaranteed to revert at enactment. Moving an operator between groups requires two correctly ordered motions, and enacting them out of order reverts the second.

Code reference: `CreateOrUpdateOperatorGroup._validateSubNodeOperators / _validateExternalOperators`.

This issue is classified as Low severity because it does not corrupt state - the motion simply cannot be enacted - but it consumes a global motion-queue slot and wastes the objection window on a motion that can never execute, per the Impact/Likelihood matrix.

Recommendation

We recommend adding membership pre-checks in the factory: in `_validateSubNodeOperators` require `metaRegistry.getNodeOperatorGroupId(noId) == NO_GROUP_ID || == groupId`, and an analogous check via `getExternalOperatorGroupId` in `_validateExternalOperators`.

Client's Commentary:

Acknowledged. We might want to be able to create motions to move a node operator from one group to another with minimal delay in between, since when an operator is not in a group, it has zero weight and is prioritized for exit. Implementing the described check would imply at least a 72-hour delay otherwise.

L-7	UpdateStakingModuleShareLimits - new share values are not bounded by TOTAL_BASIS_POINTS		
Severity	Low	Status	Fixed in b5d74557

Description

This issue has been identified within the `_validateDeltas` function of the `UpdateStakingModuleShareLimits` contract.

The factory `validates` the per-motion deltas and that `newStakeShareLimit <= newPriorityExitShareThreshold`, but it does not validate that the new values are `<= TOTAL_BASIS_POINTS` (10000). Because the share fields are `uint16`, a value in the range `10001..65535` is representable and passes the factory checks. The `StakingRouter` enforces the absolute bound at enactment (`_validateShareParams`, reverting with `InvalidStakeShareLimit` / `InvalidPriorityExitShareThreshold`), and the factory's own embedded `validateParams` re-check does not catch it either.

A motion with an out-of-range value can therefore be created and will revert at enactment. This is reachable only when `current + maxIncrease > 10000`, which depends on the deployment delta parameters; it is the only `StakingRouter` share check the factory does not mirror.

Code reference: `UpdateStakingModuleShareLimits._validateDeltas`.

This issue is classified as Low severity because it affects liveness only (an unexecutable motion) and is bounded by the configured delta limits, per the Impact/Likelihood matrix.

Recommendation

We recommend adding `require(newPriorityExitShareThreshold <= 10000)` to `_validateDeltas`; given `newStakeShareLimit <= newPriorityExitShareThreshold`, this also bounds the stake share limit.

Client's Commentary:

Fixed as of `b5d74557448a7f65ea74abdc09ff954bd113c8c3`

L-8	CreateOrUpdateOperatorGroup - no optimistic-concurrency guard allows a lost update on concurrent group motions		
Severity	Low	Status	Fixed in b5d74557

Description

This issue has been identified within the `CreateOrUpdateOperatorGroup` contract. Because `EasyTrack.enactMotion` re-invokes `createEVMScript`, the input (operator existence, `sum(share) == MAX_BP`, ordering, group-id range) is re-validated against live state at enactment, so stale references to non-existent operators correctly revert. However, unlike its sibling factories in scope - `UpdateStakingModuleShareLimits`, which pins `currentStakeShareLimit/currentPriorityExitShareThreshold` (`ERROR_CURRENT_VALUES_MISMATCH`), and `SetMerkleGateTree`, which pins `currentTreeRoot/currentTreeCid` - `CreateOrUpdateOperatorGroup` does not pin the expected current group content. As a full-overwrite operation it therefore exhibits last-write-wins behaviour: if two committee motions target the same group, the later-enacted one silently overwrites the other's update (a lost update), even though the proposer reviewed a different state.

Code reference: `CreateOrUpdateOperatorGroup.createEVMScript`.

This issue is classified as Low severity because the factory is restricted to a trusted caller, the written data is always re-validated for internal consistency, and no invalid data can be persisted; the only effect is a potential silent lost update on concurrent motions, per the Impact/Likelihood matrix.

Recommendation

We recommend optionally accepting an expected-current-group commitment (e.g. a hash of the current group) and requiring it to match the live state, mirroring the optimistic-concurrency guard used by the sibling factories.

Client's Commentary:

Fixed as of `b5d74557448a7f65ea74abdc09ff954bd113c8c3`

L-9	AllowConsolidationPair - allowlist entries are permanent and not re-validated at consolidation execution time		
Severity	Low	Status	Acknowledged

Description

`AllowConsolidationPair` enforces all consolidation constraints (operator existence, activity, and MetaRegistry group linkage) inside `_validateInputData / _validateOperatorIds`. Because `EasyTrack.enactMotion` re-invokes `createEVMScript`, these constraints are correctly re-checked against live state at both motion creation and enactment.

The gap is downstream of enactment: `ConsolidationMigrator.allowPair` itself validates nothing beyond `submitter != address(0)` and its role gate, stores the pair with no expiry, and the actual consolidation entrypoint `submitConsolidationBatch` only checks `msg.sender == submitter` and that the referenced keys are deposited. It does not re-check group linkage or operator activity.

As a result, once a pair is allowed the entry is permanent and the consolidation can be executed by the designated submitter at any arbitrarily later time. Between enactment and that execution, the operators may have been unlinked, deactivated, or moved to a different MetaRegistry group, and the submitter can still perform a consolidation that the DAO/committee no longer considers valid. The constraints the factory carefully validates can thus silently outlive the conditions under which authorization was granted.

This issue is classified as Low severity because the constraints are correctly enforced at both creation and enactment, exploitation requires a state change after enactment, and the committee can react via `disallowPair/selfDisallowPair` by monitoring `ConsolidationPairAllowed` events; the remediation, however, lies in the out-of-scope `ConsolidationMigrator`, per the Impact/Likelihood matrix.

Recommendation

We recommend re-validating group linkage and operator activity at the point of actual execution inside `ConsolidationMigrator.submitConsolidationBatch`, or introducing a time-to-live on allowlist entries, so that authorization cannot silently outlive the conditions under which it was granted. As this control lives in the migrator rather than the in-scope factory, it should be coordinated with the `ConsolidationMigrator` owners.

Client's Commentary:

Acknowledged. CMC can disallow consolidation pair if needed and the existing checks are sufficient.

3. About MixBytes

MixBytes is a blockchain security firm specializing in the analysis of decentralized protocols and smart contract systems.

The company helps Web3 teams build resilient protocol architecture, smart contract logic, and economic mechanisms through a combination of AI-assisted analysis and senior human expertise.

Rather than focusing solely on one-time audits before deployment, MixBytes works with protocols across their entire lifecycle – from early architecture design to production audits and ongoing protocol evolution.

Our team consists of experienced security researchers, engineers, and protocol analysts with deep expertise in DeFi systems, adversarial protocol analysis, and smart contract security.

Over the years, MixBytes has worked with many of the most widely used protocols in the ecosystem, including **Lido**, **Aave**, **Curve**, **1inch**, **OKX**, **Mantle**, **Fluid**, **Gearbox**, **Resolv**, and others, helping teams identify vulnerabilities, strengthen protocol design, and improve the robustness of decentralized systems.

To enhance analysis efficiency and coverage, MixBytes also develops and uses internal AI-assisted tooling that helps surface potential risk signals during development and code review. These tools augment the work of senior auditors but do not replace expert analysis.

Protocol Security Lifecycle

MixBytes supports protocols across the full lifecycle of their development and operation.

- **Design Review.** Independent assessment of protocol architecture and economic design before critical decisions are embedded in production code.
- **AI Tooling.** AI-assisted analysis integrated into development workflows to surface potential risk signals during protocol development.
- **Smart Contract Audit.** Comprehensive manual verification of smart contract logic and protocol invariants before production deployment.
- **Security Retainer.** Continuous expert support for protocol upgrades, integrations, governance changes, and evolving attack surfaces after launch.

Contacts



<https://mixbytes.io/>



https://github.com/mixbytes/audits_public



<https://x.com/MixBytes>



hello@mixbytes.io